AutoBayes brms-parity loop: Tier B review

Status at the Tier B boundary, prepared for Andrew

10 October 2026

This is the Tier B boundary review, in the same form as the Tier A one (`docs/brms-parity/reports/tier-a-review.pdf`): what landed, what parked, where the library knowingly differs from brms, how each claim was checked, what it cost, and what needs your decision before Tier C. Section 7 is the part that needs you. The acceptance rule is unchanged from Tier A: every feature merged because the AutoBayes model reproduced brms's own Stan data, log density, gradient and predictions on frozen fixtures, with the tolerances of Tier A, and nothing in any gate sampled.

1 What landed

Feature	PR	What a user can now write	Frozen brms cases it passes	Diverg.
B06 zero-inflated and hurdle Poisson	#39	`ab.zero_inflated_poisson`, `ab.hurdle_poisson`, with brms's own `zi`/`hu` and `Priors(zi=ab.Beta(a, b))`; `kind="mean"` is brms's `posterior_epred`	`zero_inflated_poisson_varint`, `hurdle_poisson_varint`	D28–D30
B02 addition terms	#40	`y | weights(w) ~ ...` for all nine Tier A families; `cens()` and `trunc()` implemented and parked (Section 2)	`gauss_weighted` and eight new `<family>_weighted` cases	D24–D26
B05 ordinal and categorical	#41	`ab.cumulative` (logit, flexible thresholds on Stan's `ordered` coordinates), `ab.categorical`; `kind="mean"` gives every category's probability	`cumulative_varint`, `categorical_poponly`, two new treatment-coded cases	D36–D39
B01 distributional formulas	#42	`ab.gaussian(["y ~ x + (1 | g)", "sigma ~ z"], ...)`; `sigma` (Gaussian, Student-t, lognormal), `shape` (negative binomial, Gamma), `phi` (Beta), log link; `predict(dpar="sigma")`	`gauss_distributional_sigma` and five new cases, one per family	D32–D35
B03 monotonic effects	#44	`mo(x)` of an ordinal predictor (whole numbers or an ordered categorical) for every family but the categorical one; `Priors(simo={"x": ab.Dirichlet(1)})`, the simplex on Stan's stick-breaking coordinates	`gauss_monotonic` and 12 new cases, one per family, plus two monotonic terms	D40–D43

How close. Measured with the gates' own comparison functions (a test plugin that records the worst error each gate sees) on the tree that merges with B03: all 72 ungated cases pass, with worst

relative errors of 8.4×10^{-10} in the log density, 7.2×10^{-9} in the gradient, 1.1×10^{-14} in predictions and 4.2×10^{-15} between the JAX and PyTensor densities. The 34 cases that became ungated in Tier B are listed in Table 1. The 38 Tier A cases are unchanged since the Tier A review, except that the log-density column now also covers the prior part (`lprior`), which G2 checks as well, so some of them read higher than in that review.

The larger numbers, traced this time. Both are on our side and both come from float32 constants that PyTensor creates without saying so.

- *Half-Student-t and half-Cauchy priors.* PyMC adds $\log 2$ to these log densities as `pt.log(2)`, which PyTensor evaluates in float32, 1.9×10^{-9} too high per prior. A 50-digit evaluation of `gauss_categorical`'s prior part puts brms within 3×10^{-15} of the exact value and AutoBayes 3.8×10^{-9} above it at every point (two such priors). This accounts for the 2×10^{-10} to 8.4×10^{-10} in the cases with these priors, including every case run under brms's default priors; the Tier A review left it untraced.
- *A fixed Student-t ν.* PyTensor makes a Python number that float32 represents exactly (4, 7, 2.5: the usual choices) a float32 constant, and the Student-t normalising constant is then computed in float32. At $\nu = 4$ each row's log density is 5.6×10^{-10} from the exact value, which is the 2.3×10^{-10} relative of the fixed-ν Student-t cases; a value such as 3.7 stays float64 and is exact.

Neither depends on the parameters, so neither changes a gradient or a posterior, and both are well inside tolerance on every frozen case. Passing ν as float64 is a one-line fix, made in pull request #45, after which the fixed-ν Student-t cases measure 3.7×10^{-16} to 5.1×10^{-16} (the table shows them before it). The $\log 2$ constant is inside PyMC itself and is listed as a follow-up. Gradient differences of 10^{-9} to 7.2×10^{-9} (Gamma shape, Beta precision, a learned Student-t ν) are probably PyTensor's C digamma (about 2×10^{-10} per evaluation, traced in Tier A) summed over rows; I have not confirmed them.

2 Parks, and one policy question

Three features are now implemented but refused, and all three for the same reason: at some frozen points brms's *own* numbers are further from the exact value than the gate tolerance, so a library that computes the exact value fails the gate. Matching brms there would mean copying its rounding error.

- **cloglog link** (Tier A, D18). brms forms $1 - \theta$ by subtraction; at one $y = 0$ row with $\eta = 3.008$ its coefficient gradient is 3.8×10^{-7} relative off the 50-digit value (tolerance 10^{-7}).
- `cens()` (B02, D24). Stan Math's `normal_lcdf` differentiates through piecewise polynomial approximations. On `gauss_censored` brms's gradient is off the 50-digit value by up to 2.5×10^{-6} relative (point 4, `sd_1`: 3.4398114909533386 against 3.4398200705418716), and five of the eight points exceed the tolerance. The B02 agent reproduced brms's numbers to 10^{-15} from Stan-Headers 2.32's own formulas. AutoBayes is within 9×10^{-15} of the reference.
- `trunc()` (B02, D25). At one point a row lies 6.17 standard deviations below its bound, where `normal_lccdf` forms $1 - \mathrm{erf}$ by subtraction: brms's gradient is off by 4.2×10^{-7}. Separately, brms's `posterior_epred` for a truncated model (the truncated mean, with $1 - \Phi$ by subtraction) is off by up to 1.05×10^{-3}. AutoBayes is within 4×10^{-14} and 8×10^{-15} of the references.

With the park lifted, both B02 cases pass every other gate (design, density, derived quantities, JAX, default priors), and `gauss_censored` also passes the prediction gate. So brms confirms the *target*; only its arithmetic in the tails is in question.

case	family	log density	gradient	predictions
bernoulli_monotonic	bernoulli	1.2e-15	1.4e-15	4.4e-16
bernoulli_weighted	bernoulli	2.1e-16	2.0e-15	3.4e-16
beta_distributional_phi	beta	8.3e-16	5.2e-10	5.1e-16
beta_monotonic	beta	6.7e-16	1.8e-10	4.4e-16
beta_weighted	beta	3.3e-16	1.2e-09	4.4e-16
binomial_monotonic	binomial	7.9e-16	4.4e-15	6.7e-16
binomial_weighted	binomial	2.8e-16	1.2e-15	8.0e-16
categorical_poponly	categorical	8.8e-16	1.3e-15	2.2e-16
categorical_poponly_categorical	categorical	3.4e-16	1.3e-15	3.4e-16
cumulative_monotonic	cumulative	1.2e-15	2.2e-15	1.4e-15
cumulative_poponly_categorical	cumulative	3.3e-16	2.7e-15	2.4e-16
cumulative_varint	cumulative	2.3e-16	6.4e-15	2.6e-16
gamma_distributional_shape	gamma	3.5e-16	7.2e-09	8.5e-16
gamma_monotonic	gamma	3.5e-16	5.9e-09	5.1e-16
gamma_weighted	gamma	2.0e-16	1.4e-09	4.6e-16
gauss_distributional_sigma	gaussian	4.0e-16	5.3e-15	2.4e-16
gauss_monotonic	gaussian	5.0e-16	2.0e-15	4.4e-16
gauss_monotonic_two	gaussian	2.1e-15	3.6e-15	1.8e-15
gauss_weighted	gaussian	3.4e-16	5.7e-15	4.4e-16
hurdle_poisson_monotonic	hurdle Poisson	6.3e-16	6.1e-15	1.7e-15
hurdle_poisson_varint	hurdle Poisson	1.6e-16	1.2e-15	4.0e-16
lognormal_distributional_sigma	lognormal	3.5e-16	1.1e-14	1.1e-14
lognormal_monotonic	lognormal	7.0e-16	2.2e-15	9.0e-16
lognormal_weighted	lognormal	3.8e-16	8.0e-15	1.0e-15
negbinomial_distributional_shape	negbinomial	3.8e-16	1.1e-10	3.9e-16
negbinomial_monotonic	negbinomial	7.0e-16	1.2e-10	2.0e-15
negbinomial_weighted	negbinomial	2.1e-16	2.8e-11	6.0e-16
poisson_monotonic	poisson	2.4e-16	6.2e-15	5.4e-16
poisson_weighted	poisson	1.5e-16	3.7e-15	4.5e-16
student_distributional_sigma	student	3.3e-16	1.8e-09	3.3e-16
student_monotonic	student	2.3e-10	5.6e-15	5.0e-16
student_weighted	student	2.5e-10	1.1e-15	4.4e-16
zero_inflated_poisson_monotonic	zero-inflated Poisson	1.1e-15	4.9e-15	8.3e-16
zero_inflated_poisson_varint	zero-inflated Poisson	2.2e-16	1.7e-15	3.9e-16

Table 1: Worst relative error against brms over the frozen points, per case that became ungated in Tier B (log density: total, without the Jacobian and the prior part; predictions: linear predictor, expected value and derived quantities).

Options.

(a) **Keep parking** (the current state). Three implemented and tested features stay refused.

(b) **Two references, both required.** For a case listed against a divergence row, each frozen point must either match brms within the usual tolerance, or else brms must differ from a committed high-precision reference by more than the tolerance *and* AutoBayes must match that reference within it. brms still certifies the target at every point where its arithmetic is accurate (most of them), and the reference certifies the numbers where it is not. This changes the gate contract (a new, opt-in path in `gates.py`, with tests that a wrong answer still fails), and these would be the first features not accepted purely against brms. The reference values would be computed by a script in the repository and committed beside the fixture.

(c) **Mirror brms's arithmetic**, so the gate passes as it stands: copies known losses of accuracy into the library on purpose. I do not recommend it.

At the Tier A boundary I recommended (a) for cloglog alone. With the same pattern three times, and with `cens()` and `trunc()` used much more often than cloglog, I now recommend (b). It is roughly one small harness change plus one reference script, and it lifts all three parks.

Splines (B04): the hard part works; integration has not started

The plan expected `s()` to park: brms builds the basis with mgcv, and nothing in our stack reproduces mgcv's thin-plate construction. Phase 1 ported it as a standalone module (`autobayes/smooths.py`; no model uses it yet, branch `parity/b04-splines`, not merged), checked against mgcv's own matrices frozen from R for 17 data sets. I regenerated all 17 from R myself: byte-identical.

- mgcv's truncated eigenbasis is not an exact eigen-decomposition. It is mgcv's own Lanczos iteration, stopped at tolerance $\epsilon^{0.7}$, so its vectors sit about 3×10^{-9} from the exact eigenvectors. A port built on an exact eigensolver would agree only to about 10^{-9}, so the port re-runs mgcv's iteration (start vector, re-orthogonalisation, convergence test and stopping point), together with mgcv's Householder sign convention, column scaling and constraint handling.
- On the 14 standard data sets (the `x` column of `gauss_spline` among them) the fixed block `Xs` agrees to 1.2×10^{-13} and the random block `Zs` agrees up to an orthogonal matrix (in practice a signed permutation) to 2.5×10^{-13}. The model is invariant under that rotation because $\mathbf{z}_s \sim N(0, I)$ is isotropic, so the adapter would map the `zs` coordinates through it; `Xs` carries a prior and must match exactly, and does. Against `gauss_spline`'s own `standata`: `Xs` to 8.1×10^{-14}, `Zs_1_1` to 6.0×10^{-15}.
- Where exactness is impossible: when the distinct values are symmetric about their centre and there are more of them than `k` (equally spaced values, or an integer covariate with every level present), a pivot in mgcv's factorisation is zero in exact arithmetic, so mgcv's basis follows rounding noise and is platform-dependent. The port refuses such data. On badly conditioned inputs (outlying values, `k` close to the number of distinct values) the agreement is as good as mgcv's own rounding allows (up to 4×10^{-10}). More than 2000 distinct values (mgcv subsamples knots with R's random number generator), other bases, `by`, and several covariates are not ported.
- A brms detail the work turned up: `brm()` gives Stan one form of `Zs` and `standata()` / `posterior_linpred` use another, 7.6×10^{-11} apart for `gauss_spline`. Its effect on the gates is about 6×10^{-12} relative in the log density, far inside 10^{-8}.

Phase 2 is the integration: `s(x)` as a term, its own PyMC wiring (Bambi has no smooth terms), the adapter's rotation mapping, and brms's default prior for `sds`. It is new spend of about one Tier B feature (0.5–0.9M agent tokens, my estimate), so I held it for you (Section 7).

3 Where AutoBayes knowingly differs from brms (Tier B rows)

The Tier A rows (D1–D23) are unchanged. Tier B added the rows below; the full text is in `docs/brms-parity/divergences.md`. Ids D27 and D31 were reserved for parallel branches and not used.

	area	brms	AutoBayes	status
D24	`cens()`	left/right/interval censoring; `normal_lcdf` by polynomial approximation	exact tails, Gaussian, codes $-1, 0, 1$	parked
D25	`trunc()`	numeric or per-row bounds; $1 - \mathrm{erf}$ by subtraction	exact tails, Gaussian, numeric bounds	parked
D26	`weights()`	zero weights allowed; `scale = TRUE`	positive finite weights; `scale` refused	kept
D28	ZI/hurdle numerics	NaN or infinite gradients for extreme η	stable forms, finite everywhere	equal on fixtures
D29	ZI/hurdle priors	any prior on `zi`/`hu`	Beta priors only, required	kept
D30	ZI/hurdle families	twelve families; formulas on `zi`/`hu`	Poisson forms with constant `zi`/`hu`	parked
D32	distributional	group terms and `offset()` in `sigma ~ ...`	population terms only	kept
D33	distributional	every parameter, any link	`sigma`, `shape`, `phi`; log link	kept
D34	distributional	`center = FALSE` still centres `sigma`'s formula	refused where Bambi cannot match	kept
D35	distributional	any column, the response included	not the response, trials or a grouping factor	kept
D36	categorical	declared factor order; `refcat`	labels sorted as R's `as.factor` sorts them; no `refcat`	documented
D37	categorical	group terms, offsets, per-category priors	population terms; one prior per class	parked
D38	ordinal	probit and other links, `disc`, `cs()`, other threshold types, `sratio`/`cratio`/`acat`	`cumulative("logit")`, flexible thresholds	parked
D39	ordinal	$P(y = k)$ by subtraction of cumulative probabilities	stable log form	equal on fixtures
D40	monotonic	interactions `mo(x):z`, `mo(x):mo(w)`	refused, naming the term	kept
D41	monotonic	`(1 + mo(x) | g)`	refused, with the population-level form	kept
D42	monotonic	`id`, expressions, `sigma ~ mo(x)`, categorical family, two-valued predictors	refused naming the construct; a two-valued predictor gets the hint to use its 0/1 column	kept
D43	monotonic	`bsp_moord`, `simo_moord1`	PyMC names `mo(ord)`, `simo_ord`; same coordinates	names only

4 How the claims were checked

- Each feature: the gate tool on the tree that merged (immutability of fixtures and tolerances, every oracle case, the full non-sampling suite), recorded with its SHA in the ledger; both hosted CI runs on the pull request; then a squash-merge as andrew-dou.
- Harness changes, each read for weakening before merging; all are extensions to new parameter kinds: B06 generalised G3's reference rule ("compare with `epred` if the log-link parent holds the

mean, else with `linpred`"), identical for every earlier case; B02's G1 compares brms's `weights`, `cens`, `lb`, `ub` with the model's vectors both ways; B05 refactored the column matcher into one function reused for the per-category designs, keeping every earlier refusal, with six new harness tests showing that corrupted thresholds, probabilities and designs fail; B01 added the distributional designs and parameters. B03 matches each monotonic term to brms's `Xmo_k` by content and maps the simplex coordinates brms's probe cannot reach in declaration order, then recomputes Stan's stick-breaking coordinates from the simplex at every point and requires them to equal brms's own unconstrained values to 10^{-10}, so a wrong assignment fails.

- Each feature: every previously ungated case's model (target identity, `describe()`, names, coefficients) snapshotted before and compared after: identical for all five merged features.
- Combined runs caught one real problem before it reached `main`: B02's registry test claimed `weights()` for every family, including B06's new ones, for which no weighted brms case exists. It now claims only the nine families with a frozen weighted case.
- Reproducibility: `oracle/regen.py check -all` regenerated every frozen fixture from R on the tree that merges with B03, 89 of them (the Tier A and Tier B cases, B03's twelve and the thirteen Tier C ones), and compared them with the committed files: all 89 identical.

Limits I know of. Some claims rest on analogy or unit tests rather than a frozen brms case. Bernoulli and binomial models take `mo()` under the probit link, but only logit cases are frozen. An ordered-categorical `mo()` predictor is checked against brms-printed codes and by building it against `gauss_monotonic`, because the oracle cannot freeze an ordered factor.

5 Process and cost

Each feature was implemented by one Opus agent in its own git worktree from a written brief, then gated, reviewed and merged by me (squash, as andrew-dou, no AI attribution). Token counts reported by the agents: B06 0.64M, B02 0.69M, B05 0.83M, B01 0.84M plus 0.11M for resolving its rebase onto B05, B04 phase 1 0.54M, B03 0.85M, and 0.43M for freezing the Tier C fixtures, about 4.9M in total, not counting my own orchestration.

That is about twice my estimate of 1.5–2M for the tier, and I flagged it to you on 10 October when it became clear. Where it went:

- Each Tier B feature cost 0.6–0.95M, against 0.3–0.6M per Tier A feature. These features needed custom likelihoods and transforms (Stan's `ordered` and simplex coordinates, the zero-inflated and hurdle forms), and several agents had to show that brms, not the library, was wrong at a frozen point, which meant 50-digit references and reproducing Stan Math's formulas.
- Stacking agents on unmerged branches of the same lane saved wall-clock time but cost rebases (B01's onto B05: 17 conflict hunks in 10 files).
- The suite takes 17–20 minutes on this machine under load, and each agent runs it at least twice.

For Tier C I would brief one feature per agent, not stack agents within a lane, and ask for the full suite once, at the end.

6 Tier C: fixtures frozen, nothing implemented

The fixtures for all seven Tier C features are frozen (pull request #43, fixture-only: 13 cases, each a strict expected failure until its feature lands). Nothing in the library has changed for them.

Feature	brms model frozen	what brms records
C01 multivariate	`mvbind(y, y2) ~ x + (1 | g)`, with and without `set_rescor(TRUE)`	one sub-model per response; residual correlation `Lrescor` (Cholesky factor); predictions with a response axis
C02 non-linear	`ypos ~ a * exp(b * x)`, `a ~ 1 + (1 | g)`, `b ~ 1`, `nl = TRUE`	`b_a`, `b_b`, a group term on `a`; non-linear parameters not centred
C03 Gaussian processes	exact `gp(ord)`; approximate `gp(x, k = 10, c = 5/4)`	`sdgp`, `lscale`, `zgp`; the approximate basis `Xgp_1` (36×10) and `slambda`
C04 autocorrelation	`ar(time = t, gr = g, p = 1)`; `arma(..., p = 1, q = 1)`	unbounded `ar`, `ma` (brms's regression form); Stan data sorted by group and time
C05 mixtures	`mixture(gaussian, gaussian)`, `y ~ x`	ordered intercepts by default, per-component `b` and `sigma`, simplex `theta`
C06 missing values, measurement error	`bf(y ~ mi(xm) + w) + bf(xm | mi() ~ w)` with 7 of 36 `xm` missing; `me(xobs, sdx)`	imputed values `Ymi_xm`; latent `Xme` as `meanme + sdme * zme`
C07 post-processing	a Gaussian and a Poisson varying-intercept model	`log_lik()`, `bayes_R2()` and `loo()` on the seeded draws; a 64-draw case so PSIS actually fits a Pareto tail ($\hat{k}$ from -0.49 to 1.74)

Two choices made while freezing. The exact GP is fitted on a four-valued input: on 36 distinct inputs its kernel matrix has condition number 10^{13}–10^{14} at the seeded points, and an independent Cholesky then differs from Stan's by 1.6×10^{-7} relative in the log density, so no implementation could meet the 10^{-8} tolerance. Several cases record fitted rows only, because brms draws random numbers for an exact GP and for `me()` on new rows; the generator now computes every prediction under two seeds and stops if they differ. CAR and SAR structures are left out (they need `data2` matrices). All 77 frozen fixtures, old and new, regenerate identically from R.

7 Decisions before Tier C

1. **The policy question** (Section 2): keep parking, two-reference acceptance (my recommendation), or mirror brms.
2. **Splines, phase 2** (Section 2): go ahead with the integration? If yes, data with symmetric knots stays refused unless you prefer a deterministic basis of our own there, documented as a divergence (it spans the same space, but its penalty scaling and coefficients differ from whatever brms produces on that platform). Porting mgcv's knot subsampling for more than 2000 distinct values I would leave out.
3. **Tier C go-ahead.** Seven features, each at least the size of a Tier B one; at Tier B's rate that is 4.5–7M agent tokens (an estimate from a tier that came in at twice my estimate). If you want less, I would do them in this order and you can stop anywhere: C07 post-processing (`log_lik`, `bayes_R2`, `loo`; probably the smallest, and used by everyone, though matching the loo package's PSIS exactly may need a port rather than ArviZ's), C01 multivariate, C02 non-linear, C05 mixtures, then C03, C04 and C06, which depend on the earlier ones.
4. **Cleanup.** Twenty-four local worktrees under `AutoBayes-worktrees/` now belong to merged or closed branches: the sixteen listed in the Tier A review, plus `docs-tierA`, `parity-b01`, `-b02`, `-b03`, `-b05`, `-b06`, `-tierC-fixtures` and `-fix-float64`. Their branches stay on GitHub. `group-exclusion` (the shared virtual environment), `parity-loopstate` (the merge-queue checkout of `main`) and `parity-b04` (spline phase 1, not merged) stay, and your original checkout is

untouched. I will remove the twenty-four only on your yes.

Follow-ups found in Tier B and not done. PyMC's half-Cauchy and half-Student-t log densities add $\log 2$ as a float32 constant (1.9×10^{-9} per prior, Section 1); the fix is either our own form of those two priors or a report to PyMC, which I would only file with your go-ahead. `weights()` for the zero-inflated and hurdle families (needs a frozen weighted case each); zero-inflated and hurdle negative binomial (D30); cumulative probit and the `sratio`/`cratio`/`acat` families (D38); group-level terms in categorical models (D37) and in distributional formulas (D32); the learned-ν `describe()` line order that depends on Python's hash seed (from Tier A); SKILL.md's front-matter description, which still says "Three supported families" and which I have not rewritten because it is your wording.